Administrator Manual

R and Python Validated Environment Setup

JR Scripts — Version 1.0

i This manual is intended for administrators only.

1. Overview

This manual describes how to set up and maintain the validated R and Python environments used by the JR Scripts project. The environment is designed to meet FDA validation requirements by using pinned package versions, a controlled local package repository, and integrity checking.

The key principle is a strict separation of responsibilities:

- The administrator sets up the local package repository and installs the environments.
- End users only run scripts — they never install packages or modify the environment.
- All packages are installed from a controlled local repository (e.g. Dropbox folder), never directly from the internet.

1.1 Architecture Overview

Component	Location	Purpose
Local R repo	R_repo/my-cran-repo	Controlled store of pinned R packages
Local Python repo	Python_repo/my-repo	Controlled store of pinned Python packages
R library	~/.renv/[PROJECT_ID]/library/	Installed R packages for all user scripts, isolated per project
Python venv	~/.venvs/[PROJECT_ID]/	Installed Python packages for all user scripts, isolated per project
renv.lock	admin/	Exact version manifest for R packages
R_requirements.txt	admin/	Human-readable pinned R package list (user-installed packages)
R_base_requirements.txt	admin/	Pinned R packages available in the R installer (base packages)
python_requirements.txt	admin/	Human-readable pinned Python package list (user-installed packages)
python_base_requirements.txt	admin/	Python packages that are part of the standard Python installer
r_version.txt	admin/	Specifies required R version (e.g. 4.5)

Component	Location	Purpose
python_version.txt	admin/	Specifies required Python version (e.g. 3.11.9)
setup_jr_path.zsh	project root	Run once by each new user to add bin/ and wrapper/ to PATH

1.2 Folder Structure

The JR environment has the following folder structure:

Folder / File	Contents
bin/	JR infrastructure scripts: jrrun, jr_versions, jr_uninstall
wrapper/	User entry point scripts: jr_animate, jr_static, and any project-specific wrappers
help/	Per-script help text files (same base name as R/Python script, .txt extension)
R/	User R scripts
Python/	User Python scripts
admin/	Admin-only scripts and configuration files (chmod 750)
admin/R/	Admin R scripts (auto-generated validation scripts)
admin/Python/	Admin Python scripts (auto-generated validation scripts)
R_repo/	Local R package repository (controlled, Dropbox-synced)
Python_repo/	Local Python package repository (controlled, Dropbox-synced)
docs/	This manual, validation report template, regulatory statement

1.3 Admin Scripts

Script	Purpose
admin/admin_install_R	Sets up the R environment from the local repo
admin/admin_install_Python	Sets up the Python environment from the local repo
admin/admin_create_hash	Generates SHA256 integrity file covering all project scripts
admin/admin_validate	Generates and runs IQ validation scripts, produces timestamped evidence file

Script	Purpose
admin/admin_uninstall	Removes the entire JR environment from this machine (admin use)
bin/jr_uninstall	Removes the current user's local environment components (renv library, venv, run log, PATH entry)
bin/jr_versions	Displays current R version, Python version, and all installed package versions
setup_jr_path.zsh	Run once by each new user — adds bin/ and wrapper/ to PATH in ~/.zprofile

2. Prerequisites

2.1 Required Software

The JR environment is intended to be installed in a shared (e.g. Dropbox) folder to which all team members have restricted access depending on their role. Before running any admin scripts, ensure the following are installed on the admin Mac:

Software	How to install
R (correct version)	Download from https://cran.r-project.org/bin/macosx/ — scroll to the required version and download the .pkg installer. Copy the installer to R_repo/. Double-click and follow the steps.
Python (correct version)	Download from https://www.python.org/downloads/macos/ — scroll to the required version and download the macOS installer (.pkg). Copy the installer to Python_repo/. Double-click and follow the steps.

2.2 Required Files

Ensure the following files are present in the admin/ folder with correct content before running any admin script:

- R_requirements.txt and R_base_requirements.txt — list of R packages with pinned versions
- python_requirements.txt and python_base_requirements.txt — list of Python packages with pinned versions
- r_version.txt — contains the required R version, e.g. 4.5
- python_version.txt — contains the required Python version, e.g. 3.11.9

2.3 Requirements File Format

Both R_requirements.txt and python_requirements.txt use the same format: one package per line, version pinned with ==, comments starting with #.

Example R_requirements.txt:

```
# R package requirements
tolerance==3.0.0
e1071==1.7-17
ggplot2==4.0.2
```

Example python_requirements.txt:

```
# Python package requirements
numpy==1.26.4
pandas==2.1.4
scipy==1.11.4
```

> ⚠ Use the exact version string as CRAN or PyPI reports it, including hyphens. For example, e1071 must be written as 1.7-17, not 1.7.17.

R_base_requirements.txt and python_base_requirements.txt list packages used in user scripts that are part of the base R or Python installation. They do not need version pins. Comments are allowed.

The requirements files shipped with the JR environment already contain the packages required by the core infrastructure scripts in `bin/`, `wrapper/`, and `admin/`. These entries must never be removed. Any packages added for project-specific user scripts should be appended below the existing entries. If a core package is accidentally removed, restore it from the original distribution or from version control before running any admin script.

3. Setting Up the R Environment

3.1 What the Admin Script Does

The script admin_install_R performs the following steps:

- Reads R_requirements.txt to get the list of pinned packages
- If called with --rebuild: downloads all packages and dependencies from CRAN into the local R repo
- Verifies the integrity of the local R repo using checksums
- Writes renv.lock from the pinned versions in R_requirements.txt
- **Installs all packages directly into the isolated R library** at ~/.renv/[PROJECT_ID]/library/ using install.packages(lib=lib_path)
- Verifies all installed versions match the requirements

> ℹ The R library is isolated per project in ~/.renv/[PROJECT_ID]/library/. It is completely separate from the system R library. Packages are never loaded from the system library during script execution.

3.2 First-Time Setup (Building the Repo)

Perform this step once when setting up for the first time, or when updating package versions. Requires an internet connection.

1. Ensure Dropbox is running and synced on your Mac.
2. Open Terminal (Applications > Utilities > Terminal).
3. Navigate to the project root folder: `cd .../jrscripts`
4. Run with --rebuild: `./admin/admin_install_R --rebuild`

When finished you should see:

```
✅ renv environment successfully created
📄 R version  : R version 4.5.0
📁 Repo       : .../R_repo/my-cran-repo
📌 Packages   : tolerance, e1071, ggplot2
```

> ℹ The first-time download may take a few minutes. Subsequent runs without --rebuild are fast as packages are already in the local repo.

> ⚠ Known limitation: --rebuild fetches the current CRAN version of dependencies, not necessarily the version specified. Always verify the installed version in the output and update R_requirements.txt accordingly. The local repository is the version control artefact for R.

3.3 Regular Setup (Installing from Existing Repo)

Use this when setting up on a new machine where the Dropbox repo already exists, or after wiping the R library.

```
./admin/admin_install_R
```

⚠ Always ensure Dropbox has fully synced before running this command. A partially synced repo will cause the integrity check to fail.

3.4 Adding a Single Package

To add one new package without rebuilding the entire repo:

```
./admin/admin_install_R --add packagename==version
```

This downloads the package and its dependencies from CRAN into the existing local repo, updates R_requirements.txt, regenerates checksums, and installs the package.

⚠ Known limitation: --add fetches the current CRAN version of dependencies, not necessarily the version specified. Always verify the installed version in the output and update R_requirements.txt accordingly. The local repository is the version control artefact for R.

3.5 Updating Package Versions

- Edit R_requirements.txt and change the version number
- Update r_version.txt if the R version also changes
- Run admin_install_R --rebuild to download the new version and rebuild the repo
- Re-generate the project integrity file (see Section 6)
- Communicate the change to all team members — the first script run will auto-rebuild their environments

⚠ Never edit the Dropbox repo files directly. Always go through admin_install_R to ensure checksums and the PACKAGES index are updated correctly.

4. Setting Up the Python Environment

4.1 What the Admin Script Does

The script admin_install_Python performs the following steps:

- Reads python_requirements.txt to get the list of pinned packages
- If called with --rebuild: downloads all packages into the local Python repo
- Verifies the integrity of the local repo using MD5 checksums
- Creates a clean Python virtual environment (venv) at ~/.venvs/[PROJECT_ID]/
- Installs all packages from the local repo using pip --no-index (no internet access)
- Verifies all installed versions match the requirements

> i Python pins versions end-to-end: pip honours exact version strings from python_requirements.txt when downloading and installing. This is more precise than the R approach — see Section 3.4 for the R limitation.

4.2 First-Time Setup (Building the Repo)

5. Ensure Dropbox is running and synced.
6. Open Terminal and navigate to the project root folder.
7. Run with --rebuild: `./admin/admin_install_Python --rebuild`

4.3 Regular Setup

```
./admin/admin_install_Python
```

4.4 Adding a Single Package

```
./admin/admin_install_Python --add packagename==version
```

pip will resolve and install all transitive dependencies at their pinned versions automatically.

4.5 Updating Package Versions

- Edit python_requirements.txt with the new version
- Update python_version.txt if the Python version also changes
- Run admin_install_Python --rebuild
- Re-generate the project integrity file
- Communicate the change to all team members

5. Environment Variables Reference

The following environment variables are set automatically by the wrapper scripts. They should not be set manually in Terminal.

Variable	Description
LOCAL_REPO	Full path to the local Dropbox R or Python package repository
BUILD_REPO	Set to true to download packages from the internet and rebuild the repo. Default false.
VENV_PATH	Full path to the Python virtual environment: ~/.venvs/[PROJECT_ID]/
RENV_HOME	Full path to the R library root: ~/.renv/[PROJECT_ID]/
RENV_PATHS_ROOT	Must equal RENV_HOME. Controls where the R library is located. Used by validate_R_env.R to locate the library.
R_LIBS	Must be unset before running R scripts. Unset automatically by wrapper scripts.
R_LIBS_USER	Must be unset before running R scripts. If set, it can redirect package loading to the system library, bypassing the validated environment.
R_LIBS_SITE	Must be unset before running R scripts.

⚠ R_LIBS_USER in particular can silently redirect package loading to the system library, bypassing the validated environment. The wrapper scripts unset this automatically — never set it manually.

6. Project Integrity Management

6.1 What the Integrity File Does

The file admin/project_integrity.sha256 contains SHA256 checksums of all project scripts. Every time a user runs a script via jrrun, the integrity file is verified before execution. If any tracked file has been modified, jrrun refuses to run and prints a clear error message.

6.2 Generating the Integrity File

Run this command from the project root folder after any legitimate change to project files:

```
./admin/admin_create_hash
```

> i Re-generate the integrity file whenever you update R_requirements.txt, python_requirements.txt, renv.lock, any R or Python script, any zsh script, or any wrapper in bin/ or wrapper/.

6.3 Repo Integrity

The local Dropbox repos have their own integrity mechanism separate from the project integrity file. When --rebuild is run, checksums of all repo files are written to checksums.txt inside the repo folder. Every subsequent install verifies these checksums.

If the repo integrity check fails:

- Check Dropbox sync status — a partial sync can cause false failures
- If files were accidentally modified, rebuild the repo with --rebuild
- Never manually edit files inside the repo folder

7. Troubleshooting

7.1 Common Errors and Solutions

Error message	Solution
✕ LOCAL_REPO environment variable is not set	Always call scripts via their zsh wrappers, not directly with Rscript or python3.
✕ Local repo not found at ...	Ensure Dropbox is installed, signed in, and fully synced. Check the repo path.
✕ Repo integrity check FAILED	Dropbox may not have finished syncing. Wait and retry. If it persists, rebuild with --rebuild.
✕ R version mismatch	Install the correct R version from https://cran.r-project.org/bin/macosx/ and ensure r_version.txt contains only the major.minor version e.g. 4.5.
✕ Version mismatch detected	The installed package version does not match requirements. Rebuild by running admin_install_R again.
✕ PROJECT INTEGRITY CHECK FAILED	A project file was modified. If the change was intentional, re-run admin_create_hash to regenerate the integrity file.
✕ renv library not found at: ...	The R library has not been built yet or was deleted. Run admin_install_R to rebuild it.
✕ RENV_PATHS_ROOT is not set	The R script was called directly, not via jrrun. Always use jrrun to execute scripts.
No module named ‘X’ (Python)	Script was called directly outside jrrun, or the venv was not built. Run admin_install_Python, then use jrrun to execute.

7.2 Checking What Is Installed

To verify which packages are installed and from where, run:

```
bin/jr_versions
```

For a full IQ validation with evidence file:

```
./admin/admin_validate
```

7.3 Starting from Scratch

If the environment is badly broken and you want to start completely fresh:

```
# Remove R library
rm -rf ~/.renv/[PROJECT_ID]
```

```
# Remove Python venv
rm -rf ~/.venvs/[PROJECT_ID]

# Rebuild both environments
./admin/admin_install_R
./admin/admin_install_Python
```

8. Adding a New Team Member

When a new team member joins, they do not run the admin scripts. The admin performs these steps:

8. Ensure the team member has Dropbox installed and has been added to the shared Dropbox folder with view-only access to R_repo/ and Python_repo/.
9. Ensure they have installed the correct R and Python versions (see Section 2.1).
10. Ask them to run `setup_jr_path.zsh` to add the JR environment to their PATH. They should drag the file to a Terminal window and press Enter, then open a new Terminal window.
11. Ask them to run `jrrun jrhello.R "Hello"` and `jrrun jrhello.py "Hello"` once. These will automatically detect that the R library and venv do not exist and build them from the local Dropbox repo.

> ℹ The first run on a new machine will take longer than usual as the R library and venv are being created. Subsequent runs are fast.

9. Access Rights

9.1 Principle

The JR environment enforces a strict separation between the administrator and end users. To make this separation enforceable at the filesystem level, project files should be owned by the admin account and made read-only for all other users. This ensures that users can run scripts but cannot modify the environment, install packages outside the validated channel, or tamper with project files in a way that would evade the integrity check.

9.2 macOS Filesystem Permissions

Location	Mode	Notes
Project root folder	755	Admin: read/write/execute. Users: read/execute only.
bin/ and wrapper/	755	Scripts executable by all users, writable only by admin.
R/ and Python/	755	Users can read scripts but not modify them.
help/	755	Users can read help files but not modify them.
admin/	750	Admin only. Users cannot read requirements files, version pins, or admin scripts.
~/.renv/[PROJECT_ID]/	755	Must remain writable by the user — the auto-rebuild writes here. See Section 9.4.
~/.venvs/[PROJECT_ID]/	755	Must remain writable by the user — the auto-rebuild writes here. See Section 9.4.

9.3 Setting Permissions After Installation

After completing the installation, run the following commands from the project root folder as the admin user:

```
# Lock down the project folder (run from project root, as admin)
chmod -R 755 .
chmod -R 750 admin/

# Remove write access for non-owners
chmod -R o-w .
```

ⓘ Re-run these permission commands after every installation update or any time new scripts are added to bin/, wrapper/, R/, or Python/.

9.4 The Auto-Rebuild Exception

The R library (~/.renv/[PROJECT_ID]/) and Python venv (~/.venvs/[PROJECT_ID]/) are stored in each user's home directory. The wrapper scripts automatically rebuild these directories when they are missing or out of date — which means the user running the wrapper must have write access to their own home directory locations.

This is by design. The auto-rebuild reads exclusively from the controlled Dropbox repo (verified by checksum) and writes only to the user's own home directory. It does not modify any project file and does not access the internet.

9.5 Dropbox Repo Access

The R_repo/ and Python_repo/ folders should be shared as view-only in Dropbox so that team members can sync them but cannot modify their contents:

- In Dropbox on the web, navigate to the shared folder containing the JR project
- Set sharing permissions for R_repo/ and Python_repo/ to Can view for all non-admin members
- The admin account retains Can edit access

> ⚠ The admin/ subfolder should never be shared with non-admin team members. If the project is in a shared Dropbox folder, ensure non-admin members have view-only access and are instructed not to access admin/.

9.6 Verifying Permissions

```
# Check project root
ls -la

# Confirm admin/ is 750
ls -la admin/

# Check bin/ and wrapper/ scripts
ls -la bin/ wrapper/
```

Expected output for bin/ and wrapper/ scripts: `-rwxr-xr-x` (owner rwx, group r-x, others r-x). For admin/ contents: `-rwxr-x---` (owner rwx, group r-x, others none).

10. Validated Environment Summary

The following table summarises all components that together constitute the validated environment for FDA purposes.

Component	Controlled by	How it is validated
R version	r_version.txt	Checked by jrrun and admin_validate before running
Python version	python_version.txt	Checked by jrrun and admin_validate before running
R package versions	R_requirements.txt + renv.lock	Verified by admin_install_R after installation; confirmed by admin_validate
Python pkg versions	python_requirements.txt	Verified by admin_install_Python after installation; confirmed by admin_validate
Package sources	Local Dropbox repo	Repo integrity checked via checksums.txt on every install
R library isolation	~/.renv/[PROJECT_ID]/library/	Explicit lib= path in install.packages(); RENV_PATHS_ROOT check at script startup
Python venv isolation	~/.venvs/[PROJECT_ID]/	pip --no-index from local repo; venv activated by jrrun
Project file integrity	project_integrity.sha256	Verified by jrrun before every script execution
Internet access	--no-index / local_repo_url only	R and Python cannot reach CRAN or PyPI during normal operation

ⓘ For FDA submission purposes, renv.lock, R_requirements.txt, python_requirements.txt, r_version.txt, python_version.txt, and the checksums.txt files inside the Dropbox repos together constitute the complete package version audit trail.

11. Adding User Scripts to the JR Environment

The JR environment hosts validated R and Python scripts used in medical device development. Because scripts are used in a regulated context, the Quality Manager must verify that documented proof of validation exists before a script is accepted. Adding scripts is the sole responsibility of the administrator.

Adding a new script involves up to three steps. Only Step 1 is mandatory.

11.1 Step 1 — Place the Script (Mandatory)

Place the R or Python script in the correct subfolder:

- **R scripts** → place in R/
- **Python scripts** → place in Python/

The script is then immediately callable via jrrun:

```
jrrun myscript.R arg1 arg2
jrrun myscript.py arg1
```

jrrun automatically routes to the correct subfolder based on the file extension. No wrapper or additional configuration is needed for the script to work.

11.2 Modifying R Scripts

Every R script must load its packages exclusively from the validated R library. Add the following block immediately before the first library() call:

```
# --- Load from validated R library ---
renv_root <- Sys.getenv("RENV_PATHS_ROOT")
if (renv_root == "") {
  stop("✕ RENV_PATHS_ROOT is not set. Run this script from jrrun.")
}
lib_path <- file.path(renv_root, "library")
if (!dir.exists(lib_path)) {
  stop(paste("✕ renv library not found at:", lib_path))
}
.libPaths(c(lib_path, .libPaths()))
```

Wrap all library() calls to suppress irrelevant startup messages:

```
suppressWarnings(suppressPackageStartupMessages({
  library(ggplot2)
  library(tolerance)
}))
```

ℹ stats and MASS are base R packages that ship with R itself. They do not need to be in R_requirements.txt and are always available.

11.3 Step 2 — Add a Help File (Optional but Recommended)

Create a plain text file in the help/ subfolder with the same base name as the script and a .txt extension:

- **R script R/myscript.R** → help file help/myscript.txt
- **Python script Python/myscript.py** → help file help/myscript.txt

When a user runs `jrrun myscript.R --help`, jrrun will display the contents of help/myscript.txt. Without this file, jrrun reports "no help file found" but still runs normally.

Suggested help file format:

```
Usage: jrrun myscript.R <input_file> <output_file>

Description:
  Brief description of what the script does.

Arguments:
  input_file    Path to the input CSV file
  output_file   Path for the output PNG file

Example:
  jrrun myscript.R data/input.csv results/output.png
```

11.4 Step 3 — Add a Wrapper (Optional)

A wrapper in wrapper/ provides a named shortcut for end users. This is optional — users can always call jrrun directly. A wrapper makes sense when:

- The script has a fixed, well-known purpose with a friendly name (like jr_animate, jr_static)
- The script takes no arguments or always the same arguments
- You want the script easily discoverable without knowing the filename

A wrapper is always a two-liner:

```
#!/bin/zsh
exec "$(dirname "$0")/../bin/jrrun" myscript.R "$@"
```

Make it executable and place it in wrapper/:

```
chmod 755 wrapper/mywrapper
```

Users can then call `mywrapper` directly from any directory, as both bin/ and wrapper/ are on the PATH.

11.5 Checking and Updating Package Requirements

Before finalising the script addition, verify that all packages used by the script are listed in the appropriate requirements file.

Package type	Where to add it
R user package (e.g. ggplot2)	R_requirements.txt using format: packagename==version
R base package (e.g. stats, MASS)	R_base_requirements.txt — no version needed
Python user package (e.g. numpy)	python_requirements.txt using format: packagename==version
Python stdlib module (e.g. os, sys)	python_base_requirements.txt — no version needed

⚠ Never add base R packages or Python standard library modules to the user requirements files. Adding them will cause installation errors.

Always use the exact version string as reported by CRAN or PyPI, including hyphens:

- R packages: https://cran.r-project.org/package=<packagename>
- Python packages: https://pypi.org/project/<packagename>

11.6 Finalising the Script Addition

After placing the script and optionally adding a help file and wrapper, always complete these steps:

12. Re-run admin_create_hash to include the new files in the project integrity file
13. If requirements files were updated: re-run admin_install_R or admin_install_Python with --rebuild, and re-run admin_generate_validate_R or admin_generate_validate_Python
14. Inform team members. If package versions changed, their environments will auto-rebuild on next run.

⚠ Always re-run admin_create_hash after adding or modifying any script, wrapper, or help file. If you forget, the integrity check will fail for all team members on their next run.

12. Run Log

12.1 User Run Log

Every time a user runs a script via jrrun, a log entry is appended to:

```
~/.jrscript/[PROJECT_ID]/run.log
```

Each entry contains: ISO timestamp, script name (jrrun), script filename and arguments, exit code. Example:

```
2026-03-12T14:32:01     jrrun jrhello.R Hello   0
2026-03-12T14:35:44     jrrun jrhello.py Hello  0
```

12.2 Admin Log

Admin actions (admin_install_R, admin_install_Python, admin_validate) are logged to:

```
~/.jrscript/[PROJECT_ID]/admin.log
```

Format is the same as the run log with the admin script name and exit code or PASSED/FAILED status.

> ℹ For FDA audit purposes, the run log and admin log together provide a complete record of all script executions and administrative actions. Both logs are per-project and per-user.

13. Important Note on Validation Scope

The validation evidence supplied with this distribution is specific to the R version, Python version, and package versions present at the time of release. Revalidation is required whenever:

- The R or Python version is changed in r_version.txt or python_version.txt
- Any of the package version numbers as shipped is updated in R_requirements.txt or python_requirements.txt
- The environment is installed on a new machine as part of a regulated workflow

Revalidation is performed by executing the Validation Plan in `docs/templates/validation_plan_template.docx` and recording results in a new Validation Report. The original validation evidence remains on file as the baseline for the released version.